+34 691 225 633
Rev. Febrero 2024

Autor: Jefferson David Rodríguez Chivatá

Matplotlib & Seaborn: Guía básica

Introducción

Bienvenidos a esta guía básica, revisaremos dos de las bibliotecas de visualización de datos más poderosas y populares en Python; Matplotlib & Seaborn. En esta guía, nos enfocaremos en introducir algunas de las gráficas básicas vistas en el tema principal que pueden crear utilizando estas herramientas, abriendo la puerta a un mundo de posibilidades en el análisis y presentación de datos. Desde gráficos de líneas y barras hasta visualizaciones de dispersión y más. También, aprenderán cómo estas bibliotecas pueden transformar números crudos en representaciones visuales claras y atractivas, además de cuando usar cada una, ventajas y desventajas.

Contenido

1. Revisión general de Matplotlib & Seaborn:

En Python, dos librerías se sitúan como pilares en el mundo de la visualización de datos: Matplotlib, Seaborn. Cada una contribuye de forma única al ecosistema de la visualización, ofreciendo un conjunto de herramientas diversas para los científicos de datos. Existen otras librerías de visualización de datos en Python, como Plotly o Bokeh, que se revisarán en las siguientes clases. Pero, todo se inicia con las proporcionadas anteriormente.

- **Matplotlib**:

En el núcleo de la visualización de datos en Python se encuentra Matplotlib, una biblioteca versátil que permite crear visualizaciones estáticas, animadas e interactivas. La importancia de Matplotlib radica en su flexibilidad y ubicuidad. Como biblioteca básica, proporciona una base sólida para la creación de visualizaciones básicas, al tiempo que ofrece la complejidad necesaria para el trazado avanzado.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

- **Seaborn**:

Seaborn, que se integra perfectamente con Matplotlib, añade una capa de sofisticación estadística a sus visualizaciones. Destaca en la presentación de datos sutiles, ofreciendo una gama de tipos de gráficos adaptados al análisis estadístico. Esta librería simplifica la creación de visualizaciones complejas, proporcionando un puente entre los datos en crudo y la información procesable.

2. Introducción a la graficación en Matplotlib & Seaborn.

En este apartado vamos a adentrarnos en la implementación de algunos de los gráficos explicados anteriormente. El objetivo es brindar las primeras herramientas para la construcción de gráficos básicos en Matplotlib. Tenga en cuenta que, iremos aumentando el nivel para construir gráficas más complejas. Además, incorporaremos Seaborn para evidenciar sus ventajas sobre Matplotlib en la creación de gráficos más estadísticos.

- **Matplotlib**:

Usualmente Matplotlib es algo complejo de aprender para los usuarios nuevos. Sin embargo, abordaremos conocimientos iniciales para que el aprendizaje sea mucho más rápido. Lo primero que debemos conocer es que Matplotlib tiene dos interfaces de uso: Explícita e Implícita. La interfaz explícita es cuando directamente usamos una instancia del objeto `figure.Figure`, el cual se puede ver como un lienzo que contiene ejes (`axes.Axes`) y es donde se visualizan las gráficas. Por otro lado, existe la interfaz implícita, que es la que usaremos en los ejercicios de este documento. Esta interfaz está inspirada en MATLAB y es una interfaz global encapsulada en el módulo `pyplot` el cual grafica en los ejes que estén siendo usados en el momento. Otros puntos importantes a tener en cuenta, es que la apariencia de las gráficas de Matplolib por defecto no son lo suficientemente llamativas. Sin embargo, la librería cuenta con una amplia variedad de configuraciones para mejorar la apariencia.

Gráficos de líneas y barras en Matplotlib: Crear gráficos de líneas y barras utilizando Matplotlib de manera implícita es una tarea sencilla. Primero, consideremos un escenario donde se tiene información de la temperatura de una

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

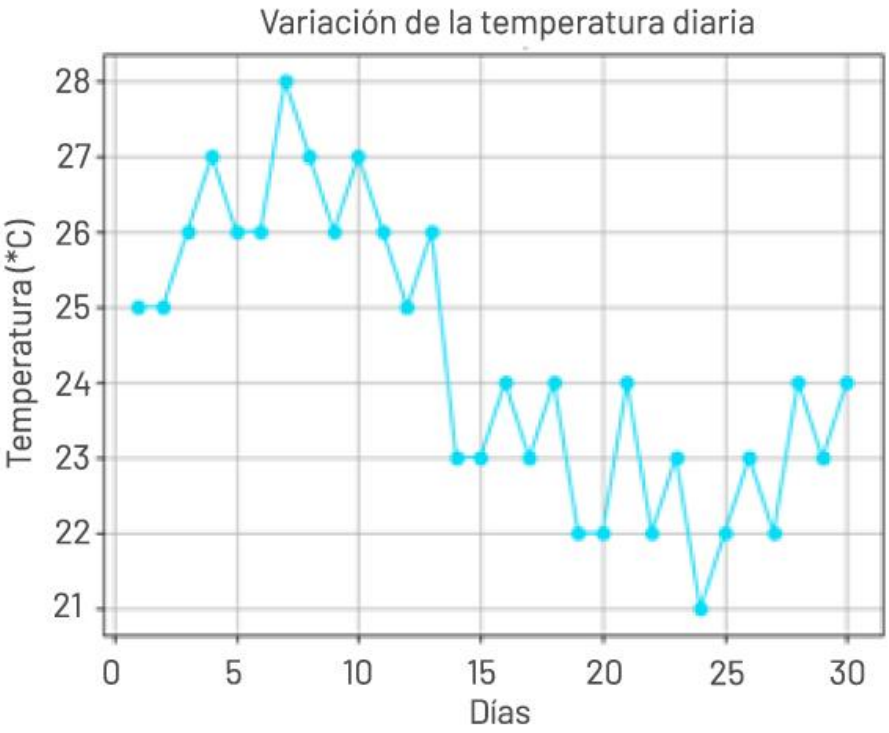
ciudad para todos los días del mes. Como se enseñó podemos usar en este caso un gráfico de línea para visualizar los datos, por lo tanto:

```
import matplotlib.pyplot as plt

# Definir los datos (x = días)
days = [ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17,
       18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30]
# Crear la temperatura para cada día (y = temperatura)
temperature = [25, 25, 26, 27, 26, 26, 28, 27, 26, 27, 26, 25, 26,
23, 23, 24, 23, 24, 22, 22, 24, 22, 23, 21, 22, 23, 22, 24, 23, 24]
# Crear el gráfico de línea
# Es posible cambia el estilo de la línea y el marcador a usar en
cada punto
plt.plot(days, temperature, marker='o',
         linestyle='-', color='skyblue')
# Agregar título y etiquetado de ejes
plt.xlabel('Días')
plt.ylabel('Temperatura (°C)')
plt.title('Variación de la temperatura diaria')
# Mostrar la grilla de líneas y la gráfica
plt.grid(True)
plt.show()
```

Es importante evidenciar como se importa el módulo pyplot del cual podemos construir la gráfica de líneas directamente. Además, se usaron propiedades de personalización como estilo de línea y marcador de cada dato. Puede revisar los siguientes links para conocer los tipos de líneas y marcadores disponibles además de los colores (Estilos de línea, Marcadores, Colores). Las opciones de personalización dependen del tipo de gráfico, sin embargo, hay opciones globales que modifican toda la apariencia de los gráficos. La gráfica resultante:

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 10: Gráfico de línea en Matplotlib



Fuente: Elaboración propia.

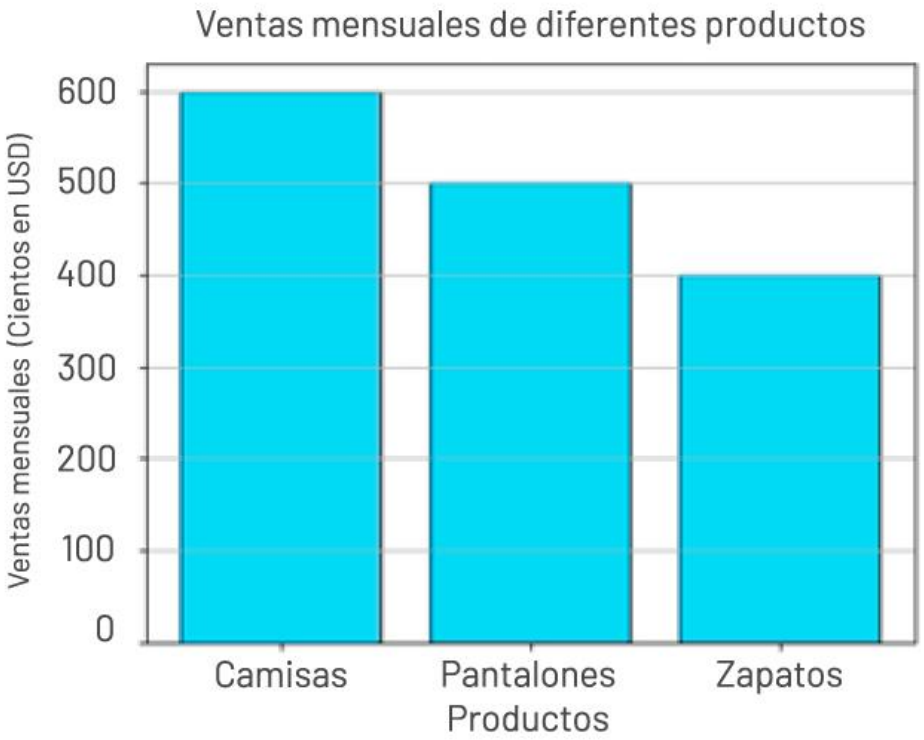
Consideremos ahora otro escenario hipotético en el que tenemos datos de ventas mensuales de diferentes productos. Para lo cual, un gráfico de barras es ideal. El siguiente fragmento de código ilustra cómo construir un gráfico de barras:

```
import matplotlib.pyplot as plt # Método implicito

# Datos de ejemplo (Parejas de datos para cada barra)
products = ['Camisas', 'Pantalones', 'Zapatos']
sales = [600, 500, 400]

# Creación del gráfico de barras y características visuales
plt.bar(x=products, height=sales,color='skyblue',edgecolor='black')
plt.xlabel('Productos') # Etiqueta eje x
plt.ylabel('Ventas mensuales (Cientos en USD)') # Etiqueta eje y
plt.title('Ventas mensuales para diferentes productos') # Titulo
plt.grid(axis='y') # Activar lineas horizontales
plt.show() # Mostrar la gráfica
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

El código anterior genera la siguiente figura de barras:

Figura 11: Gráfico de barras en Matplotlib



Fuente: Elaboración propia.

Gran parte de los tipos de gráficas básicas disponibles en Matplotlib siguen la estructura de las gráficas anteriores (Ver tipos de gráficas). Sin embargo, a medida que debemos hacer gráficas más complejas, el código necesario para construirlas empieza a incrementar, y es aquí cuando el uso de Seaborn es una gran ayuda, ya que simplifica la construcción de muchas gráficas. Considere el siguiente ejemplo de una gráfica de barras apiladas en Matplotlib.

```
import matplotlib.pyplot as plt

# Datos sobre la cantidad de productos vendidos por trimestre
products = ['Laptop Dell', 'Laptop ASUS', 'Laptop Apple', 'Laptop
Lenovo']
quarter1_sales = [150, 200, 100, 120]
quarter2_sales = [180, 220, 120, 150]

# Para Crear las barras agrupadas es necesario indicar la posición
# en el eje x, se le suma el ancho de la barra a las posiciones
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

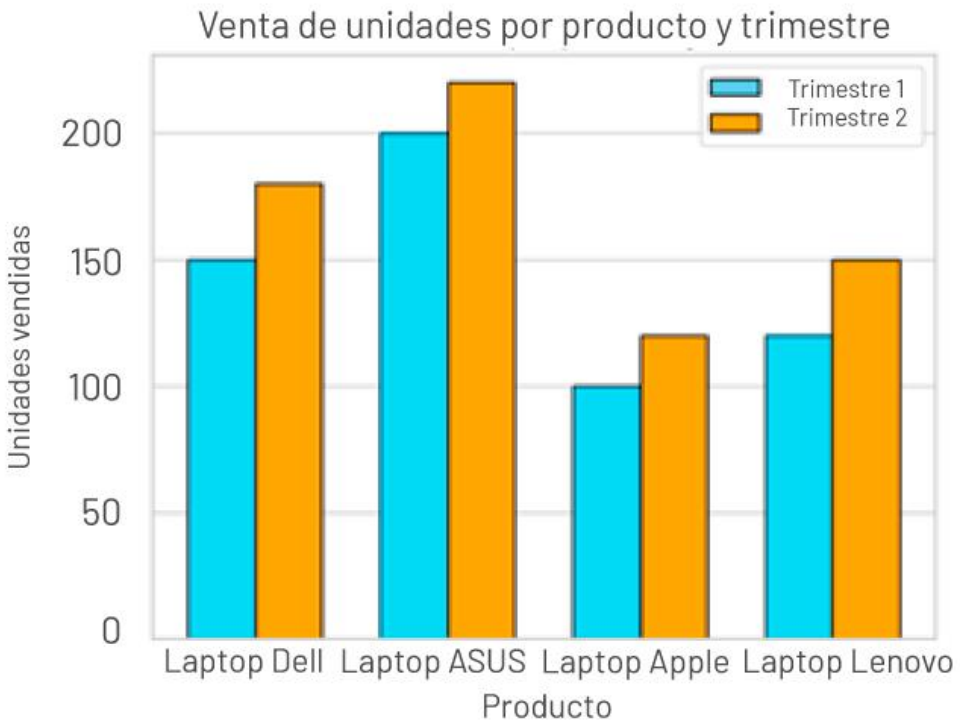
+34 691 225 633


```
# del primer grupo
bar_width = 0.35
r1 = [0, 1, 2, 3]
r2 = [0.35, 1.35, 2.35, 3.35]

# Es decir se crean dos graficos de barras pero matplotlib
# los une en un solo gráfico
plt.bar(r1, quarter1_sales, color='skyblue',
        width=bar_width, edgecolor='black', label='Trimestre 1')

plt.bar(r2, quarter2_sales, color='orange',
        width=bar_width, edgecolor='black', label='Trimestre 2')

# Añadir título y etiquetado de ejes
plt.xlabel('Producto')
plt.ylabel('Unidades vendidas')
plt.title('Venta de unidades por producto y trimestre')

# Hay que posicionar los nombres de cada producto en la mitad
# de las dos barras para cada grupo
plt.xticks([r + bar_width/2 for r in range(len(products))],
products)
# Dibujar la leyenda para cada trimestre
plt.legend()

# Mostrar las lineas horizontales de la grilla y la gráfica
plt.grid(axis='x')
plt.show()
```

Es evidente notar que la cantidad de código para crear una gráfica de barras sencilla y una anidada cambia en gran proporción. Esto es debido a que Matplotlib se enfoca en ser una librería genérica que permita desarrollar y personalizar cualquier tipo de gráfico. Sin embargo, precisamente esto resulta ser una desventaja en algunas ocasiones.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 12: Gráfico de barras agrupadas en Matplotlib



Fuente: Elaboración propia.

Ahora que hemos comprendido cosas básicas de Matplotlib podemos abordar el uso de Seaborn para optimizar y agilizar el tiempo en la creación de gráficas con requerimientos complejos y funcionalidades más estadísticas.

- **Seaborn**:

Seaborn es una librería para hacer gráficos estadísticos en Python. Se basa en matplotlib y se integra estrechamente con las estructuras de datos de Pandas. Sus funciones de trazado realizan internamente el mapeo semántico necesario y la agregación estadística para producir gráficos informativos (Consulte aquí los tipos de gráficas). Veamos algunos ejemplos:

Gráficos para la visualización de distribuciones: Recordemos que si deseamos visualizar la distribución de datos podemos usar histogramas y gráficas de

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

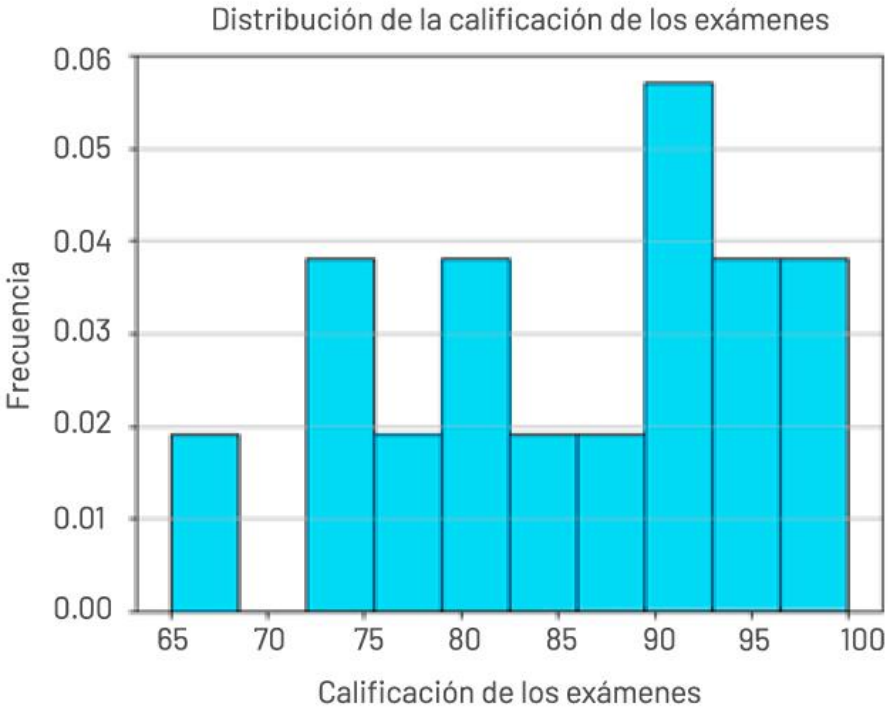
densidad, entre otros. Considere el siguiente escenario donde usaremos Matplotlib y Seaborn para crear un histograma y un gráfico de densidad sobre la información de las calificaciones de 15 estudiantes. Seaborn facilita la construcción de gráficos orientados al análisis estadístico, en este caso las gráficas de densidad.

```
import matplotlib.pyplot as plt

# Datos del puntaje de los exámenes para 15 estudiantes
exam_scores = [65, 75, 80, 85, 90, 95, 87, 76, 92, 95, 98, 100, 74,
81, 90]
# Crear el histograma con matplotlib
# Bins significa el número de grupos que usará para agrupar los
datos
plt.hist(x=exam_scores, bins=10, color='skyblue',edgecolor='black')
plt.xlabel('Calificación de los exámenes')
plt.ylabel('Frecuencia')
plt.title('Distribución de la calificación de los exámenes')
plt.grid(axis='y') # Activar líneas horizontales
plt.show()
```

En los histogramas hay un parámetro que es importante y animamos a explorarlo siempre. Dicho parámetro son los Bins en los que se van a agrupar los datos para contabilizar las frecuencias. Dependiendo del número de Bins escogido, nuestra gráfica puede presentar variaciones visuales. Por lo que se recomienda probar con diferentes Bins para encontrar el que más se adecue a nuestros datos. El código anterior genera la siguiente gráfica:

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 13: Histograma en Matplotlib



Fuente: Elaboración propia.

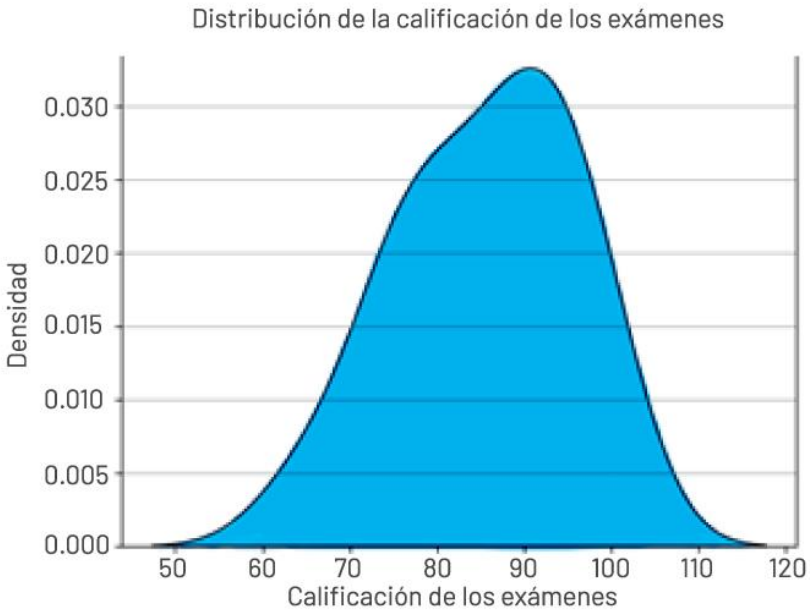
Se observa como Matplotlib agrupa la información de las calificaciones en 10 barras. Ahora, otra manera de visualizar esta distribución, pero esta vez de manera continua, es usando una gráfica de densidad. Veamos como con Seaborn podemos conseguirlo:

```
import seaborn as sns
import matplotlib.pyplot as plt

# Datos del puntaje de los examenes para 15 estudiantes
exam_scores = [65, 75, 80, 85, 90, 95, 87, 76, 92, 95, 98, 100, 74,
81, 90]

# Crear el gráfico KDE (Kernel density estimation) usando Seaborn
sns.kdeplot(exam_scores, fill=True, color='skyblue',
edgecolor='black')
plt.xlabel('Calificación de los exámenes')
plt.ylabel('Densidad')
plt.title('Distribución de la calificación de los exámenes')
plt.grid(axis='y') # Activar lineas horizontales
plt.show()
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Este tipo de gráfica en Matplotlib, hubiese requerido mucho más código. Sin embargo, Seaborn lo simplifica y genera la siguiente gráfica:

Figura 14: Gráfico de Densidad en Seaborn



Fuente: Elaboración propia.

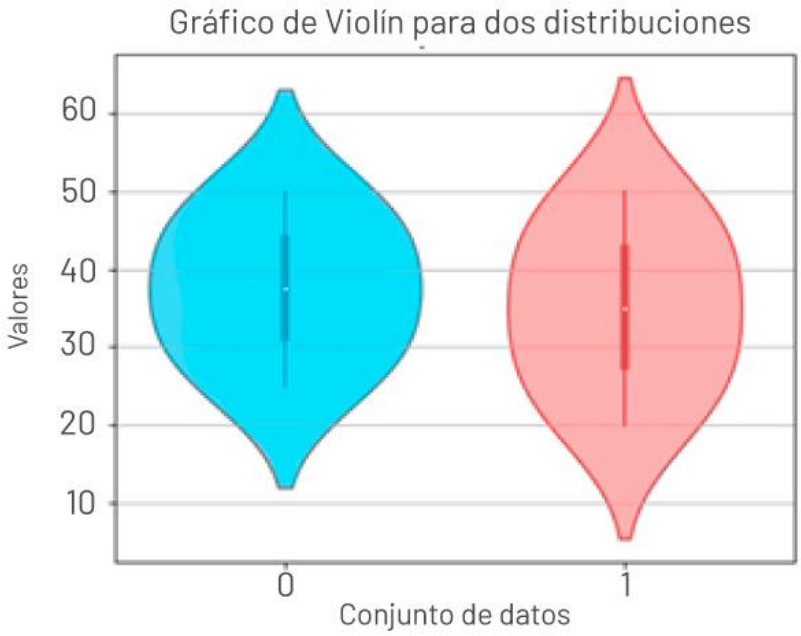
De manera complementaria, Seaborn nos permite crear gráficos para comparar varias distribuciones en una misma gráfica. En este caso es posible usar gráficos de cajas, violín entre otros. El siguiente ejemplo muestra un caso donde tenemos dos conjuntos de datos y queremos comparar sus distribuciones:

```
import seaborn as sns
import matplotlib.pyplot as plt

# Puntos de datos para dos datasets
data_set1 = [25, 30, 35, 40, 45, 50]
data_set2 = [20, 25, 30, 35, 40, 45, 50]

# Crear el gráfico de Violin
sns.violinplot(data=[data_set1, data_set2], palette='pastel')
plt.xlabel('Conjunto de datos')
plt.ylabel('Valores')
plt.title('Gráfico de Violin para dos distribuciones')
plt.grid(axis='y')
plt.show()
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Así de sencillo es crear un gráfico para comprar varias distribuciones en un mismo gráfico. Revisemos la visualización resultante:

Figura 15: Gráfico de Violín en Seaborn



Fuente: Elaboración propia.

Para seguir explorando Seaborn, vamos a usar un gráfico muy útil para explorar relaciones entre variables: Gráfico de Burbuja.

Gráfico de burbuja para representar relaciones de 3 variables: Consideremos un escenario en el que queremos explorar la relación entre dos variables, "X" e "Y", al tiempo que consideramos la magnitud de una tercera variable, "Z". En este hipotético conjunto de datos, "X" e "Y" representan coordenadas en un espacio bidimensional, y "Z" significa una magnitud asociada a cada punto de datos:

- **Eje X:** Representa una variable de interés, quizás las horas dedicadas a actividades relacionadas con el trabajo.
- **Eje Y:** Representa otra variable, como la puntuación de productividad alcanzada durante esas horas de trabajo.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

- **Eje Z (Tamaño de las burbujas):** Representa una tercera dimensión, por ejemplo, el valor monetario de los proyectos completados durante esas horas de trabajo.

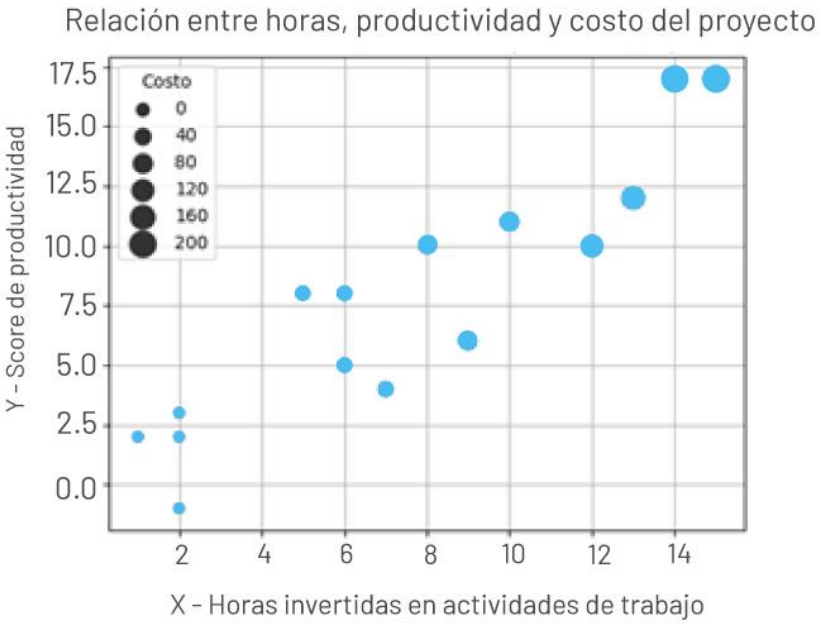
Visualicemos ahora este escenario mediante un gráfico de burbujas, en el que cada punto del gráfico no sólo capta la relación entre "X" e "Y", sino también la magnitud denotada por el tamaño de la burbuja.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Datos de ejemplo para cada variable
X = [2, 7, 2, 6, 14, 6, 10, 5, 13, 1, 12, 8, 9, 2, 15]
Y = [2, 4, 3, 5, 17, 8, 11, 8, 12, 2, 10, 10, 6, -1, 17]
Z = [1, 48, 1, 33, 189, 38, 98, 29, 179, 2, 139, 65, 82, 0, 223]

# Crear el diccionario de datos
data = {'Horas': X, 'Productividad': Y, 'Costo': Z}

# Crear el gráfico de burbuja
# Sizes especifica el rango del tamaño de las burbujas
sns.scatterplot(x='Horas', y='Productividad', size='Costo',
data=data, sizes=(50, 230), legend=True)

# Añadir título y etiquetado de ejes
plt.xlabel('X - Horas invertidas en actividades de trabajo')
plt.ylabel('Y - Score de productividad')
plt.title('Relación entre horas, productividad y costo del
proyecto')

# Mostrar las lineas de la grilla y la gráfica
plt.grid()
plt.show()
```

El parámetro “Size” en este caso siempre será el que no permite visualizar la relación de la tercera variable, con la variable “legend” se activa una leyenda a un lado de la gráfica que muestra la escala para esta variable. Recuerde usar el rango para la variable “size” ya que con respecto a este fija el tamaño de las burbujas. El resultado correspondiente es el siguiente:

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 16: Gráfico de Burbuja en Seaborn



Fuente: Elaboración propia.

Así, es como podemos visualizar las relaciones entre 3 variables, si solo quisiéramos revisar dos variables sería un diagrama de dispersión.

Conclusiones

Familiarizarse con las librerías esenciales de visualización como Matplotlib y Seaborn: Recuerde en qué momento usar cada opción para ahorrar tiempo, siempre use Matplotlib para personalizar sus gráficas, una buena presentación añade bastante valor al resultado final. Como Tip importante, Seaborn está construida sobre Matplotlib y está pensado para trabajar con DataFrames de pandas, domine esta herramienta ya que las visualizaciones de Seaborn trabajan con estructuras en formatos “long”.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

+34 691 225 633
Rev. Febrero 2024

Referencias

- Moffitt, C. (2017). Practical Business Python, Effectively Using Matplotlib. Recuperado el 20 de noviembre de 2023 de https://pbpython.com/effective-matplotlib.html

- Waskom, M. (n.d.). An Introduction to Seaborn. Recuperado el 20 de noviembre de 2023 de https://seaborn.pydata.org/tutorial/introduction.html

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.